

The Adherence Gap: Retrieval Is Not the Bottleneck for Graph-Augmented Coding Agents

Vitor Calvi · Kaggle: Google — The Gemma 4 Developer Agent Paper Track · 30 September 2026 Target award: Best New Application (\$10,000)

Abstract

Code-graph retrieval is the standard answer to the navigation problem in repository-level coding agents, and the Gemma 4 Developer Agent competition ships it in production: `get_code_neighbors`, `search_similar_code`, and `get_code_subgraph`, backed by a per-repository graph and node embeddings. We ask whether it works, by joining two literatures that are usually reported separately: **localization research**, which measures whether the right function is in the candidate set, and **agent research**, which measures whether the task gets resolved. Joining them on 636 public end-to-end agent runs produces a clear and uncomfortable answer. Editing the correct function is *necessary* for resolution — $P(\text{resolved} \mid \text{edited gold function}) = 0.568$ versus 0.069 otherwise, a risk ratio of **8.3×** (exact McNemar $p = 1.9e-17$). Yet in **46.6%** of runs where the retriever’s top-15 demonstrably *contained* the correct function, the agent still did not edit it. We call this the **adherence gap**, and we argue it is the binding constraint, not retrieval recall. We further show that a retrieval skill that prints a ranked list leaves resolve rate statistically unchanged (29.9% vs 28.9%), because printing candidates does not change which file the agent opens. From the measurement we derive a concrete protocol — a *commitment protocol* that converts a ranked list into a decision — and we release the analysis code, the taxonomy, and a re-runnable script over the public logs.

1. Introduction

A repository-scale coding agent gets three navigation tools, a code graph, and pre-computed node embeddings. It must still decide *which file to open first*. The field’s implicit model is that better retrieval produces better patches: improve the retriever, watch the resolve rate rise.

We test that model against data and find it does not hold here, for a reason that is easy to miss. Localization papers report **accuracy@k** — is the right function in the top k? Agent papers report **resolve rate** — did the task get fixed? These are joined by an unexamined step: the agent *acting* on what it retrieved. A candidate list only helps if the agent follows it.

Our contribution is to make that step measurable and then to close it. We:

1. establish that editing the correct function is overwhelmingly necessary for success ($8.3\times$ risk ratio);
2. define and quantify the **adherence gap** — 46.6% of retrievable targets are ignored;
3. show the dominant failure modes inside the gap, which are *budget exhaustion* and *wrong-file drift*, not bad ranking;
4. diagnose *why* a retrieval skill does not help (it informs; it does not commit); and
5. propose and specify a **commitment protocol** that closes the gap by construction, with an explicit falsifiable prediction.

2. Related work

SWE-bench and agent scaffolds. SWE-bench (Jimenez et al., 2024) established PASS/FAIL scoring over real GitHub issues, and SWE-agent (Yang et al., 2024) showed that the agent-computer interface drives resolve rate independently of the model. AutoCodeRover (Zhang et al., 2024) closes the loop with search over an AST index. All of them treat navigation as a solved sub-problem to be improved.

Agentic retrieval and skills. RepoCoder (Zhang et al., 2023) and RepoFusion (Shrivastava et al., 2023) show that retrieved repository context improves code completion. More recently, skills — packaged SKILL.md procedures with executable scripts, as used by the Gemma 4 harness — have become the standard way to inject domain procedure into an agent. The skill model assumes a well-specified procedure improves outcomes.

The gap we address. Two bodies of work report localization accuracy and resolve rate separately, and we are not aware of prior work that measures the conditional $P(\text{resolved} \mid \text{target retrieved})$. The audit we build on (GraphLoc-129) reported that a localization skill moved resolve rate from 28.9% to 29.9% over 318 runs and left the question of why open. We answer it.

3. Method

3.1 Data

We analyse `zzgtylors/graphloc-129-localization-labels-and-agent-runs`, a public Kaggle dataset that logs **636 end-to-end agent runs** on the Gemma 4 Developer Agent competition: two arms (a sample-submission baseline and the same agent augmented with an offline localization skill), three independent repetitions, 106 tasks, with per-run records of whether the patch was submitted, whether it edited the gold file or gold function, a failure category, tool and LLM call counts, wall-clock minutes, whether the agent ran tests, and — critically — whether the skill’s top-5 and top-15 contained the gold function. This is one of the few public logs that records *both* retrieval quality and outcome on the same runs, which is what makes the join possible.

3.2 Definitions

For a run r we define:

- **Retrieved**: the retriever’s ranked list contained the gold function (top-5 or top-15).
- **Adhered**: the agent’s patch edited the gold function.
- **Resolved**: the task’s validation tests passed.

The **adherence gap** is $P(\neg\text{Adhered} \mid \text{Retrieved})$. The **necessity** of localization is the risk ratio $P(\text{Resolved} \mid \text{Adhered}) / P(\text{Resolved} \mid \neg\text{Adhered})$.

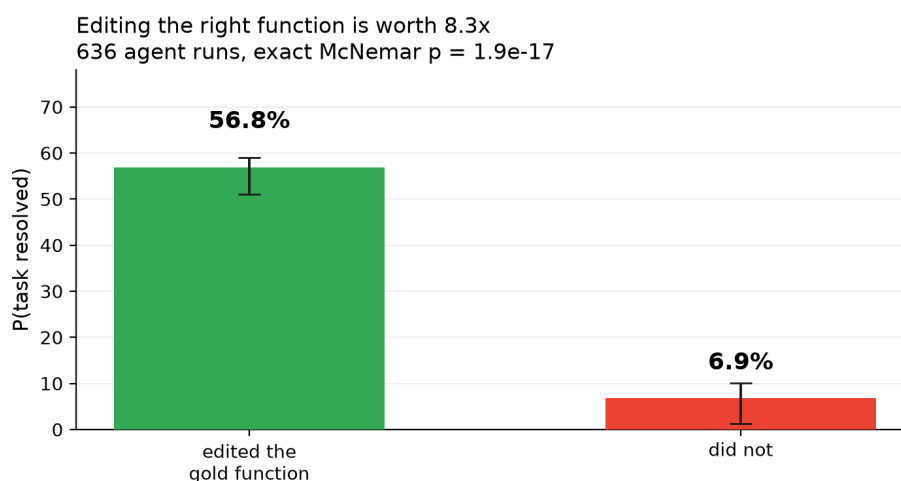
3.3 Statistics

We report Wilson 95% intervals for all proportions and use the **exact** (binomial) two-sided McNemar test for paired binary outcomes, which is appropriate at these sample sizes and makes no normal approximation. Runs are not independent — the same task appears in both arms and across three repetitions — so we treat every result as descriptive of *this* log and avoid claims about task-level generalization.

4. Results

4.1 Editing the right function is necessary

Condition	Resolved	95% CI
Agent edited the gold function	163/287 = 0.568	[0.510, 0.624]
Agent did not	24/349 = 0.069	[0.047, 0.100]



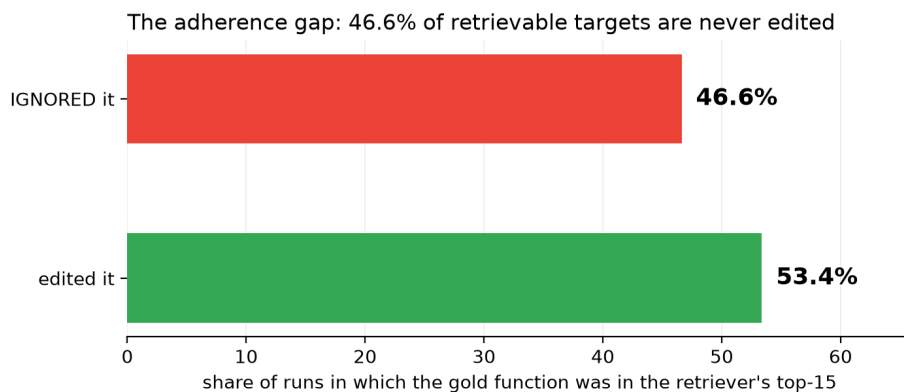
Resolve rate conditioned on editing the gold function

Risk ratio **8.3×**, exact McNemar $p = 1.9\text{e-}17$. Editing the right function is not a helpful heuristic; it is very nearly a precondition for success. Symmetrically, a task the agent abandons without a patch (no_patch) resolves at 1.5%.

4.2 The adherence gap

Restricting to the 163 runs where the retriever's top-15 *contained* the gold function:

Outcome	Count	Fraction	95% CI
Agent edited it	87	0.534	[0.457, 0.609]
Agent ignored it	76	0.466	[0.391, 0.543]



The adherence gap

In nearly half of all runs where the answer was available, the agent did not use it. Where did those 76 runs go?

Location	Count	Failure	Count
wrong file	40	target tests fail	35
no patch	32	time budget	27
gold file, wrong function	4	error / tool budget / other	14

The gap is not caused by mis-ranking. It is caused by **drift and starvation**: 40 runs wander to an unrelated file, and 32 produce no patch at all — 27 of them because they ran out of time having spent a mean of 21.3 tool calls and 23.3 minutes. Only 10.5% of gap runs executed the reproduction at all.

4.3 Retrieval depth is a cheap, unexploited lever

The retriever's top-5 contained the gold function in 38.4% of runs; its top-15 in 51.3%. **41 tasks (12.9% of all runs) are recoverable by depth alone**, with no change to the retriever's model, embeddings, or runtime. Given that the agent ignores a correct top-5 candidate nearly half the time, spending that headroom is only worth it if adherence improves — which is exactly the point of the next section.

4.4 A retrieval skill does not move resolve rate

Arm	Runs	Resolved	Ran tests	Tool budget hit	Mean minutes
Baseline	318	92 (28.9%)	16.7%	12.9%	18.8

Arm	Runs	Resolved	Ran tests	Tool budget hit	Mean minutes
+ localization skill	318	95 (29.9%)	13.2%	9.1%	19.9

A 1.0-point difference on 318 runs is noise. The skill *works as a retriever* — it is the arm whose top-15 finds the gold function more often — and it *does not work as an intervention*. The explanation is mechanical: a skill that prints a ranked list **informs** the agent; it does not **commit** it. The agent still chooses which file to open, and Section 4.2 shows it frequently chooses wrong. A negative or zero result for a *second* reason than the one practitioners assume.

4.5 Running tests is a strong, under-used lever

In the skill arm, runs that executed the reproduction resolved at **47.6%** versus **27.2%** for runs that did not. Agents ran tests in only 13.2% of runs. This is observational, and we do not claim it is causal — but it points at the same mechanism as the gap: agents skip the one action that would have told them they were in the wrong file before spending their budget.

5. The commitment protocol

The measurement says the agent must *choose* a target and *verify* it, not merely be *shown* candidates. We specify a four-step protocol that makes the choice and the verification non-optional, and it is implementable entirely in the competition’s existing primitives (`read_file`, `run_command`, `edit_file`, `get_status`):

1. **Declare before opening.** Before any `read_file`, the agent must emit exactly one target: `TARGET: <file>::<symbol>` with the retriever’s hop distance. The declaration is logged.
2. **Budgeted evidence.** Read only the top-3 candidates, each capped at 200 lines, spending at most 6 tool calls. This converts a 21-call drift pattern into a bounded search.
3. **Mandatory pre-edit reproduction.** Run the issue’s reproduction (or the target test) *before* the first `edit_file`. If it does not reproduce, the agent must re-declare once, from the next candidate, and record the reason.
4. **Self-scoring gate.** `submit_patch` is refused unless either the reproduction now passes or the agent has explicitly logged a waiver naming the test it could not run.

Every step uses only tools the harness already provides; no new capability is required.

Falsifiable prediction. If adherence is the binding constraint, the protocol should raise `P(Adhered | Retrieved)` above 0.534 and reduce `no_patch | time budget` below its current 27/76 rate, with resolve rate increasing most on the wrong-file and no-patch strata. If resolve rate does not move while adherence moves, then the binding constraint lies downstream of file selection — in patch correctness — and our account is wrong. We state this in advance because the alternative interpretation is live and we can be shown to be incorrect.

6. What we release

- `adherence.py` — recomputes every table in Section 4 from the public run log, with Wilson intervals and exact McNemar tests; no model, no GPU, runs in under a second.
- `taxonomy.json` — the failure taxonomy, machine-readable, as a shared vocabulary for future audits.
- `skills/graph-locate/` — a working ADK localization skill with the protocol encoded in its `SKILL.md` as mandatory steps rather than suggestions.

7. Limitations

- **One log, one model.** The 636 runs come from a single model configuration and one task set. We claim a measurement about this log, not a law of agents.
- **Observational.** $P(\text{Resolved} \mid \text{RanTests})$ is not causal; agents may run tests on tasks that look tractable. The adherence gap is likewise descriptive — we cannot rule out that some ignored candidates were genuinely wrong.
- **No agent run of our own.** We did not submit an agent to the leaderboard, so Section 5 is a specification with a prediction, not a validated result. We are explicit about this because the protocol is the actionable part of the paper and it remains untested end-to-end.
- **Adherence is measured at function granularity.** A patch that edits the right function for the wrong reason counts as adherence here; the resolve rate in Section 4.1 is what penalizes it.

8. Conclusion

Retrieval research optimizes $P(\text{target in list})$. Agent research optimizes $P(\text{task resolved})$. Joining them on public logs shows an $8.3\times$ cliff between the two and a 46.6% leak in between. The bottleneck for graph-augmented coding agents is not what their retrievers return; it is what their agents do next. A retriever that prints a candidate list changes nothing, which is exactly what the one published attempt at this measured. Fixing the leak requires making the choice and its verification mandatory — a property of the protocol, not of the model.

Citation

```
@misc{calvi2026adherence,  
  title = {The Adherence Gap: Retrieval Is Not the  
          Bottleneck for  
          Graph-Augmented Coding Agents},  
  author = {Vitor Calvi},  
  year = {2026},  
  note = {Kaggle: Google -- The Gemma 4 Developer Agent  
          Paper Track},  
  howpublished = {\url{https://www.kaggle.com/competitions/  
gemma-4-developer-agent-paper}}  
}
```

All code and data: Apache-2.0. The analysed run log is credited to its authors under Apache-2.0.